

1. Problem
2. Input Sanitization
3. Escaping
4. Encoding
5. HTML Sanitization

## Input Sanitization

Input sanitization means cleaning user input before showing it or using it in the application.

User can put anything:

- HTML tag
- Javascript Code
- Malicious URL

Problem:

- XSS attack
- Page break
- Data or Session theft

1. Escaping
2. Encoding
3. HTML Sanitization

## Escaping

Convert special HTML characters into plain text so the browser displays them instead of interpreting them.

don't use innerHTML

Use `textContent`

```
< -> &lt;
> -> &gt;
> & -> &amp;
> ' -> &#39;
> " -> &quot;
```

```
<script>alert('Hello')</script>
```

```
&lt;script&gt;alert('Hello')&lt;/script&gt;
```

## Encoding

Encoding converts data into another representation so it can be safely transmitted or displayed.

Hello World —————> Hello%20World

Space - %20

- URL: (/, space)
- Query String - ?, &, =
- headers - base64
- json - "/"

## HTML Sanitization

HTML sanitization means removing dangerous parts from HTML while keeping safe formatting.

```
<p>LoopKaka<b>Channel</b></p>
<script>alert("abc")</script>
```

Output:

```
<p>LoopKaka<b>Channel</b></p>
```

Rich-text editor - CKEditor

User Comment

Email

## Libraries:

Angular, Recat, Vue etc

JS - DOMPurifier

Java, Paython, .net - (html-sanitizer)

